NAME:

STUDENT ID:

Remember to also initial in the upper right-hand header of each page.

Due Friday Nov. 6th by the end of class.

Portfolio 8: Files

To succeed in these problems you will need to read and engage with the embedded activities in the **Runestone texbook chapter** (worth 0.5 of a grade percentage point):

- 11: Files

Note that these activities are focused on this week's theme, but they are designed to utlize skills from all your previous portfolios as well. You should use your judgement to decide which tasks are best solved with which skills you have accumulated so far.

For completing and self-correcting 70% of the below activities, you will earn 1 grade percentage point. Additionally, for participating in a *share-out of 1 of the activities, you will earn 0.5 of a grade percentage point*.

Note that some activities we will plan to start in class and others you need to start on your own. You should finish each activity before its in-class share-out in order to correct it before final submission.

Submission: To get credit for completing a prgramming activity, submit it to gradescope and answer the written questions about your algorithm, whether your program worked and what troubleshooting you can think of. You *only have to print programs you want my feedback on*. Printed programs can still be marked up by hand for syntactic or semantic corrections.

Table of Contents

This portfolio includes the following 10 activities and the report of your weekly share-out:

Part	Problem/Task	Page	In-class Start	In-class Share-out	Submission Format
A1	Written prompt	2	F 10/30	F 10/30	
A2	Prog. with Python (describe data)	3	M 11/2	M 11/2	GS, print
A3	Prog. with Python (make randoms)	4	W 11/4	W 11/4	GS, print
A4	Prog. with Python (translation)	5	oyo	W 11/4	GS, print
A5	Prog. with Python (author id, freq)	7	Th 11/5	Th 11/5	GS, print
A6	Prog. with Python (author id, prob)	8	Th 11/5	Th 11/5	GS, print
A7	Prog. with Python (author id, mse)	9	Th 11/5	Th 11/5	GS, print
A8	Written	11	oyo	F 11/6	written
A9	Written	12	oyo	F 11/6	written
A10	Prog. with Python (word list)	13	oyo	F 11/6	GS, print
B1	Share-out report	14	oyo		written
B2	Share-out report (optional)	15	oyo		written

A1) Prompt: Does the ability to use files change your opinion on whether computers can think? Does having long-term memory affect how humans can think?**

A2) Files I/O: Describe Data

- Download the starter file `random_numbers.txt` from the course website.

1. Write a function called `describe_data` that takes the name of a text file containing some data in the form of one number per line (for example, the `random_numbers.txt` file you've just downloaded) and prints out the following information about the data:

```
the filename
how many data points the file contains
the average
the minimum
the maximum
```

- The output produced by `describe_data` should have the following format. The exact values for the filename, number of data points, average, minimum, and maximum will vary depending on the data file being read. Feel free to use built-in list or string methods for this assignment.

```
file: some_data_file.txt
N: 100000
Average: 50.0494
Min: 1
Max: 100
```

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

For feedback, insert you printed program(s) here.

A3) Files I/O: Make Randoms

- Take a look at the starter file `random_numbers.txt` from the previous activity.

1. Create a new function `make_randoms`.The function should write the specified number of randomly numbers to a file called `new_random_numbers.txt` with one number printed per line. The function should take three parameters:

    - the number of random numbers to generate
    - the smallest number to generate, and the
    - largest number to generate

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

For feedback, insert you printed program(s) here.

A4) Files with dictionaries for translation

0. Download the starter file `eng-ger.txt`. This file shows examples of how English words are mapped to German words. Also create a file called `translation.py`.
1. Write a function called `read_word_dict` that reads in the word file, stores the entries in a dictionary that maps English words to German words and returns the dictionary. For example:

```
>>> read_word_dict("eng-ger.txt")
{'growls': 'knurrt',is: 'ist','barks': 'bellt',
'dog': 'hund', 'sleeping': 'schlafend', 'at': 'an',
'the': 'der', 'cat': 'katze'}
```

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

2. Add a function `nice_print_dict` to `translation.py` that prints a nice display of the dictionary to the screen. (Hint: a for-loop over a dictionary iterates over its *keys* by default.) For example:

```
>>> translation_dict = read_word_dict("eng-ger.txt")
>>> nice_print_dict(translation_dict)
the.................der
dog.................hund
barks...............bellt
cat.................katze
is..................ist
sleeping............schlafend
growls..............knurrt
at..................an
```

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

3. Write a function called `translate` that takes an English sentence (represented as a string) and returns a word-by-word translation of the sentence into the other language. For example:

```
>>> translate("the dog barks", td)
'der hund bellt '

>>> translate("the cat is sleeping", td)
'der katze ist schlafend '

>>> translate("the dog barks at the cat", td)
'der hund bellt an der katze '
```

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

4. Prepare a second, similar text file containing English words and their translation into another language. (Choose enough words so that you can use them to form a few sentences). The format of the file should be the same as the example provided. That is, each line of the file should contain one entry in the format English word: foreign word.

You can write and look at text files (ending in .txt) using IDLE. You just have to make sure to choose text files from the drop down menu in the window for saving or opening files. IDLE only shows you Python files by default.

Remember to test out the functions you've written on this new translation file.

For feedback, insert you printed program(s) here.

A5) Author Identification: Frequency Dictionary

This is the first part of a multi–part assignment in which you'll write a program that helps identify the author of a piece of text.

1. Download the following files from the course website:
   a. `author_identification.py`: the starter file
   b. `state-union-gwbush-minus-2006.txt`: all State of the Union addresses by George W. Bush except for the 2006 one. To make later processing more convenient, all punctuation has been stripped and words converted to all lowercase.
   c. `state-union-obama-minus-2014.txt`: all State of the Union addresses by Barack Obama except for the 2014 one. To make later processing more convenient, all punctuation has been stripped and words converted to all lowercase.
2. Build a word-frequency dictionary. Write a function called `make_word_frequency_dictionary` that takes the name of a file and returns a dictionary that maps words to how often they occur in the text.

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

  .....

For feedback, insert you printed program(s) here.

A6) Author Identification: Probabilities

- besure to complete the previous activity first

1. Go from word-frequencies to word-probabilities. Write a function `make_word_probability_dictionary` that takes a word-frequency dictionary and creates a dictionary that maps words to probabilities, instead of raw frequency counts.

That is, the dictionary you are building should contain the same words as the word-frequency dictionary. To get the probabilities of the words, you have to divide the frequency of each word by the total number of words found in the text. Note that this is not the number of different words in the text (word types), but the total number of word tokens. In other words, if a word appears multiple times, it gets counted multiple times. One way of getting the total number of words in the text is to sum up all values in the word-frequency dictionary.

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

2. Why do humans write (and speak) in different, individual ways? Does the way someone writes and/or speaks affect how they think?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

For feedback, insert you printed program(s) here.

A7) Author Identification: Mean Squared Error

- You'll need your code (`author_identification.py`) and the two text files from the previous parts of Author Identification.
- Download the following files from the course website:

  a. `mystery1.txt`: a State of the Union address
  b. `mystery2.txt`: a State of the Union address
  c. `mystery3.txt`: a State of the Union address

One of the three mystery files is the 2006 State of the Union address by George W. Bush, one is the 2014 State of the Union address by Barack Obama, and one is a State of the Union address by another president.

1. General approach: Your goal is to write a program that can help you guess which of the mystery texts is by Bush, which is by Obama, and which is by somebody else (without you having to actually read the texts yourself). To figure out who, most likely, wrote each of our mystery texts, we are going to compare each mystery text to the state of the Union addresses of George W. Bush and Barack Obama. If the mystery text is more similar to one than to the other, we have some evidence as to who composed it.

To measure how similar two texts are, we are going to build word-probability dictionaries for both texts (using the functions you implemented in previous parts of this assignment). Then we are going to measure the difference between those two dictionaries.

As a measure of difference, we are going to use the mean squared error. That is, for each word, we are going to calculate the difference between the probabilities provided by the two dictionaries, then we are going to square that difference. We sum up the squared differences for all words and, finally, compute the average of them. The mean squared error is a measure of difference (i.e. the smaller the mean squared error, the less different / more similar two texts are).

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

2. Is your program more of less accurate than you thought it would be? Explain what is or is not surprising to you about this task

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

For feedback, insert you printed program(s) here.

A8) Written Questions

Assume that the following function definitions for sum are used with the provided scores array. For each definition say (a) what gets printed to the screen and (b) briefly describe the purpose of the function.

```
scores =    [[1, 2, 3],
             [4, 5, 6],
             [7, 8, 9]]
print(sum(scores))
```

1.
```
def sum(grid):
    sums = []
    for row in grid:
        rowsum = 0
        for num in row:
            rowsum += num
        sums += [rowsum]
    return sums
```

.....

.....

.....

2.
```
def sum(grid):
    sums = [0] * len(grid)
    for row in grid:
        for i in range(0, len(row)):
            sums[i] += row[i]
    return sums
```

.....

.....

.....

3.
```
def sum(grid):
    total_sum = 0
    for row in grid:
        for num in row:
            total_sum+= num
    return total_sum
```

.....

.....

.....

A9) More Written Questions

Arrange the following lines of code into a program that reads text in from a file and computes the average length of the words in the text. The text file contains multiple lines and each line consists of several words.

```
01. def avg_word_length(filename):
02. avg_word_length("dracula.txt")
03. return sum_word_lengths / word_count
04. pipe = open(filename, "r")
05. words = line.split()
06. word = word.strip(".,;:!?-\"'\n\t ")
07. word_count = 0
08. sum_word_lengths = 0
09. word_count += 1
10. sum_word_lengths += len(word)
11. for word in words:
12. for line in pipe:
13. if len(word) != 0:
14. pipe.close()
```

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

A10: Word List from a File

0. Download the starter files `short_file.txt` and `dracula.txt` from the course website.
1. Write a function called `create_wordlist_from_file` that takes the name of a text file and returns a list of all words in the text.

One strategy you can use for dividing the text file up into words: assume that all words are separated by white space characters (e.g. "space", "tab", "new line"). This assumption will lead to some "words" that include punctuation characters (e.g. "Bistritz.–Left"). This is perfectly acceptable. For example, giving the function `dracula.txt` should produce a word list that starts as follows:

```
['3', 'May.', 'Bistritz.--Left', 'Munich', 'at', '8:35' ...]
```

After defining your functions, add the following line to your code:

```
### DO NOT DELETE THIS LINE: beg testing
```

Test your function definition. Make sure to leave some evidence of testing in your file. Make sure to add appropriate comments.

2. (Optional) Try to improve your function so that punctuation characters are not included in words.

- In english (not code), what is your algorithm for your solution?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

  .....

For feedback, insert you printed program(s) here.

B1: Share-out

0. Did you share-out this week? If not, and if you would like to make up your credit, where in the upcoming schedule would you like to be added?

   .....

   .....

   .....

   .....

1. For which activity did you share-out?

   .....

   .....

   .....

   .....

2. What was the solution you shared?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

3. If any, what changes did you make to your solution in response to questions or discussion?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

B2: Share-out (optional)

0. Did you share-out this week? If not, and if you would like to make up your credit, where in the upcoming schedule would you like to be added?

   .....

   .....

   .....

   .....

1. For which activity did you share-out?

   .....

   .....

   .....

   .....

2. What was the solution you shared?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

3. If any, what changes did you make to your solution in response to questions or discussion?

   .....

   .....

   .....

   .....

   .....

   .....

   .....